

Lecture

Problems in sequence processing

Problem	Input	Output	Relation	Mathematical formulation
Next word prediction	Text	Next word	Many-to-one	$p(y_t y_{t-1}, y_{t-2}, \dots, y_1)$
Sentiment classification	Text	Label	Many-to-one	$p(c y_t, y_{t-1}, \dots, y_1)$
Machine translation	Text in one language	Text in other language	Many-to-many	$p(y_t, y_{t-1}, \dots, y_1 x_t, x_{t-1}, \dots, x_1)$

Representations vs features

- Feature vector is a representation
- Representations are usually not interpretable
- Representations capture hidden structure, more than we can describe with features

Representations vs features

Consider next word prediction:

The most complicated and difficult part of it was only just **beginning**

Idea 1: use previous word (or words)

Feature representation: [0, 0, 0, 1, 0, 0]

Problem 1: long-range dependencies

France is where I grew up, but I now live in Boston. I speak fluent ____

Idea 2: use bag-of-words (TF-IDF) Feature representation: [0, 0, 2, 0, 3, 0]

Problem 2: order is not preserved

The food was good, not bad at all
vs
The food was bad, not good at all

Sequence processing models requirements

- Processes variable-length sequences
- Captures long-term dependencies
- Retain order information
- Natural preprocessing

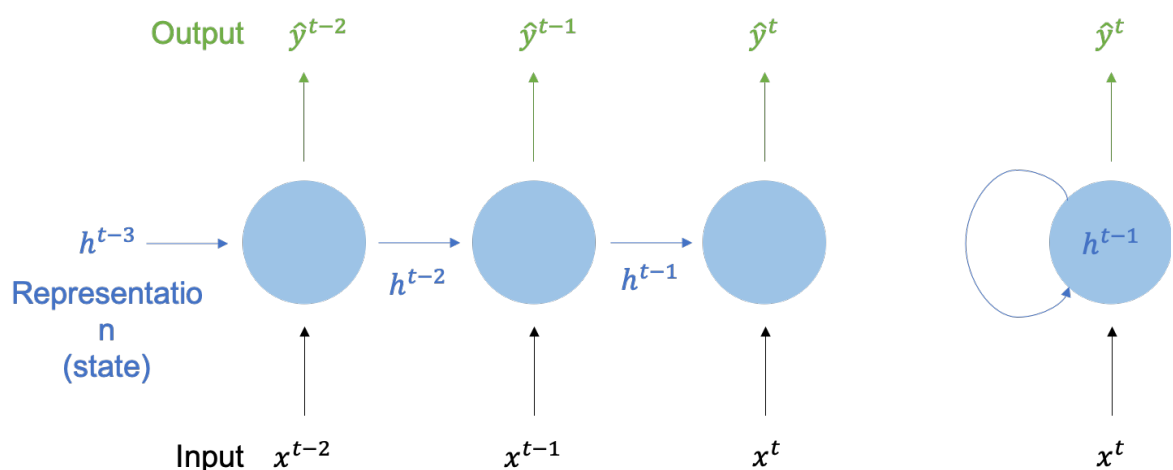
Sequence processing models: classic ML

Processes variable-length sequences	Limited
Captures long-term dependencies	No
Retain order information	No
Natural preprocessing	kinda

Sequence processing models: classic FCNN

Processes variable-length sequences	Limited
Captures long-term dependencies	Limited
Retain order information	Limited
Natural preprocessing	kinda

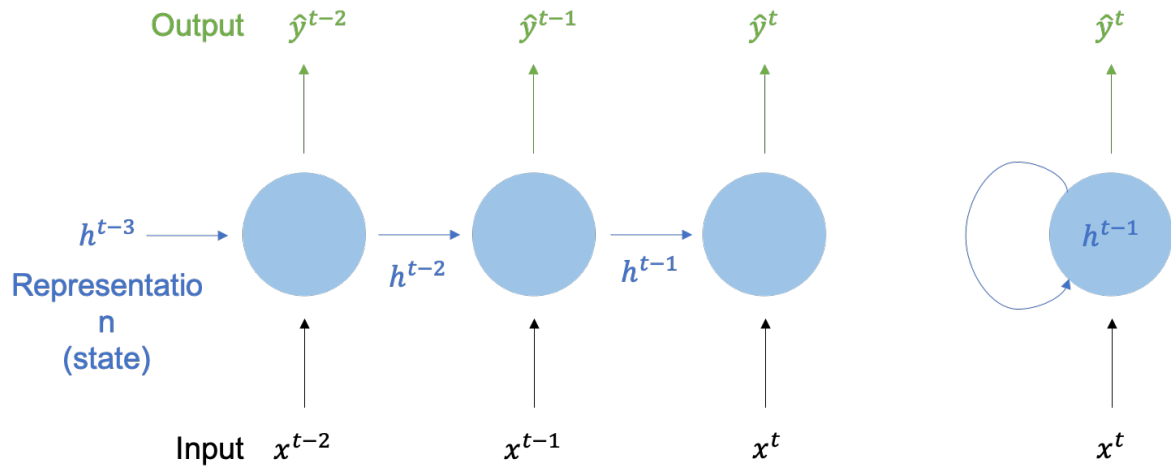
Solution: Recurrent Neural Network



$$h_t = f_h(h_{t-1}, x_t), \hat{y}_t = f_y(h_t)$$

$$p(y_t | y_{t-1}, y_{t-2}, \dots, y_1, x_t, x_{t-1}, x_{t-2}, \dots, x_1) = p(y_t | h_{t-1}, x_t) = p(y_t | h_t)$$

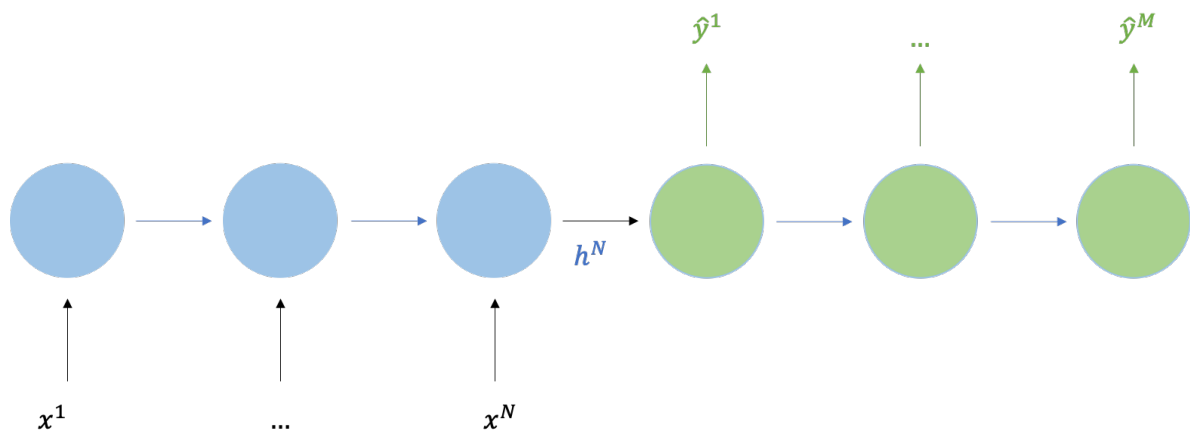
Vanilla RNN



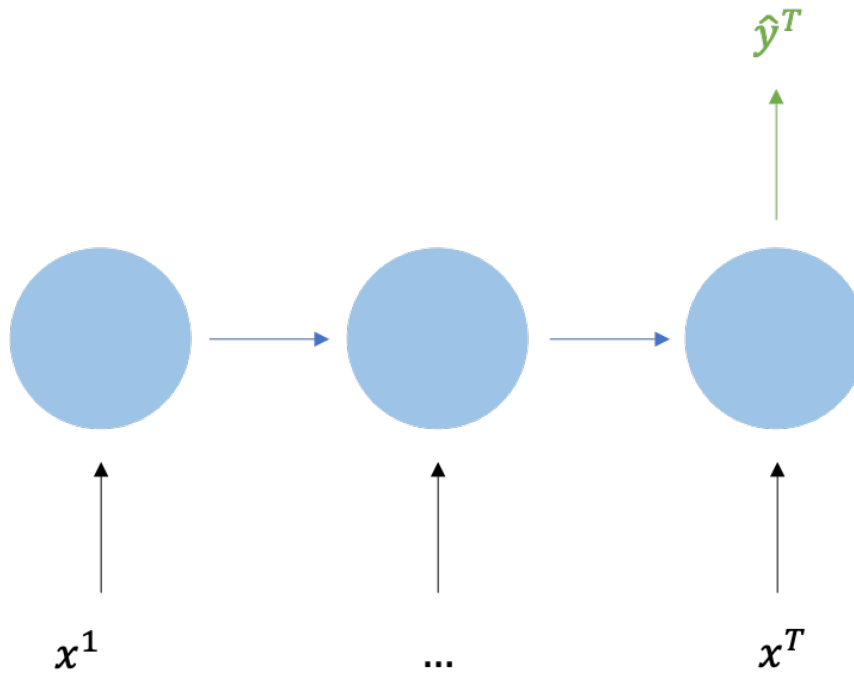
$$h_t = \tanh(Vx_t + Wh_{t-1} + b_h),$$

$$\hat{y}_t = \text{softmax}(Uh_t + b_h)$$

Sequence to sequence model



Problems with RNN: long inference



Processing time is $O(T)$

Problems with RNN: gradients

$$h_t = \tanh(Vx_t + Wh_{t-1} + b_h),$$

$$\hat{y}_t = \text{softmax}(Uh_t + b_y)$$

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial W}$$

$$\frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial W} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h^t} \left(\frac{\partial h^t}{\partial W} + \frac{\partial h^t}{\partial h^{t-1}} \frac{\partial h^{t-1}}{\partial W} + \dots \right)$$

$$\frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial W} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h^t} \sum_{i=0}^t \frac{\partial h^i}{\partial W} \left(\prod_{j=i+1}^t \frac{\partial h^j}{\partial h^{j-1}} \right)$$

Problems with RNN: exploding gradients

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h^t} \sum_{i=0}^t \frac{\partial h^i}{\partial W} \left(\prod_{j=i+1}^t \frac{\partial h^j}{\partial h^{j-1}} \right)$$

If $\left\| \frac{\partial h^j}{\partial h^{j-1}} \right\| > 1$, then non-stationary. Gradients can "explode" and become nans.

Solutions:

- Gradient clipping

- BPTT

Problems with RNN: vanishing gradients

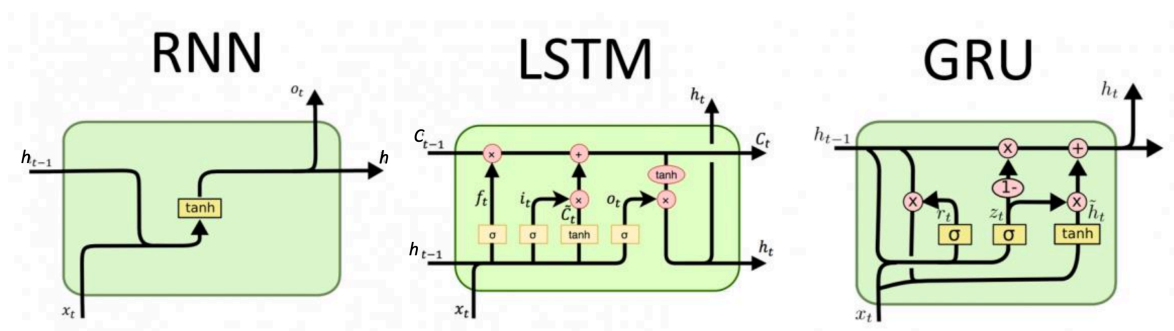
$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \sum_{i=0}^t \frac{\partial h^i}{\partial W} \left(\prod_{j=i+1}^t \frac{\partial h^j}{\partial h^{j-1}} \right)$$

If $\left\| \frac{\partial h^j}{\partial h^{j-1}} \right\| < 1$, then gradients quickly decay to zero with t , can't learn long-range dependencies. Solutions:

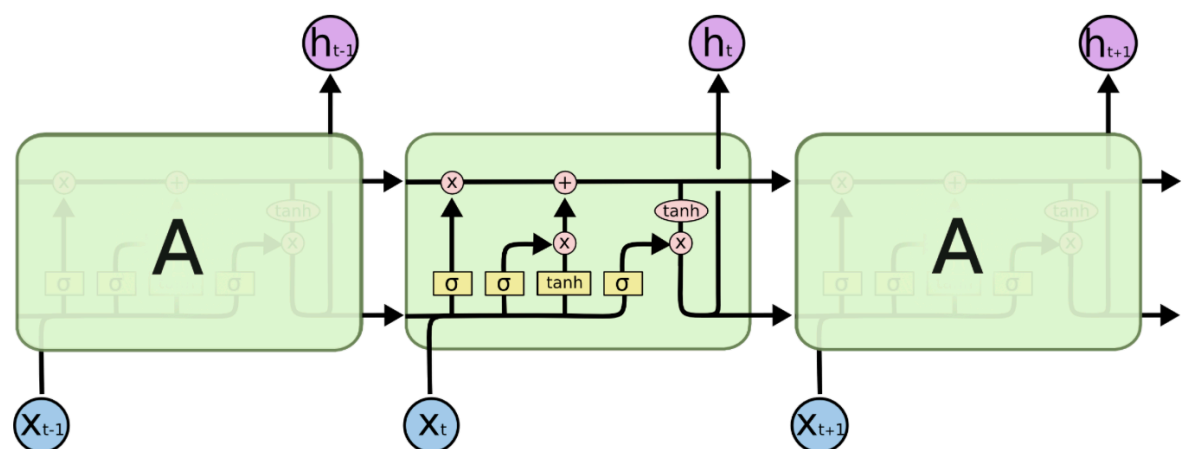
- Use ReLU
- Clever initialization of parameters

Because of this, vanilla RNNs have a fundamental problem of quick forgetting.

Solution: gated architectures



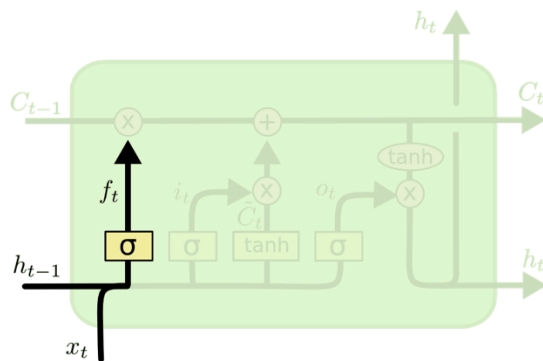
LSTM



The key to LSTMs is the introduction of cell state. Temporal representation now consists not only of hidden state (capturing short-term dependencies), but also of the cell state, capturing long-range dependencies. LSTM has the ability to change

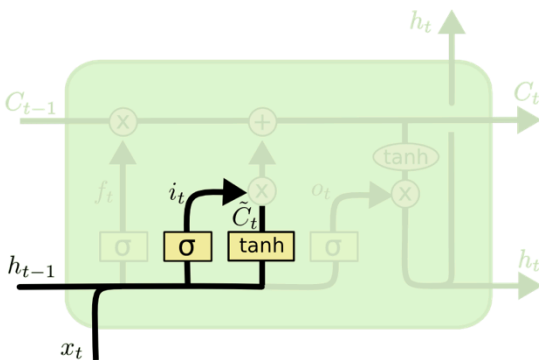
the cell state with the information from hidden state, regulated by three gates, forget gate, input gate and output gate.

LSTM: forget gate



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

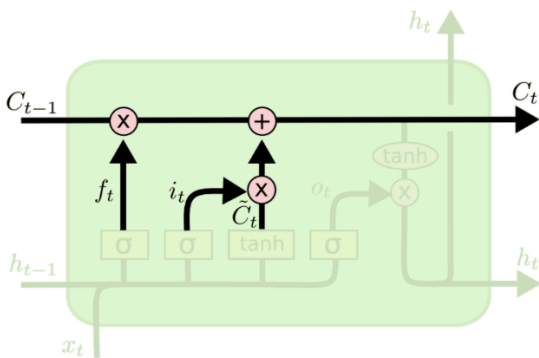
LSTM: input gate



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

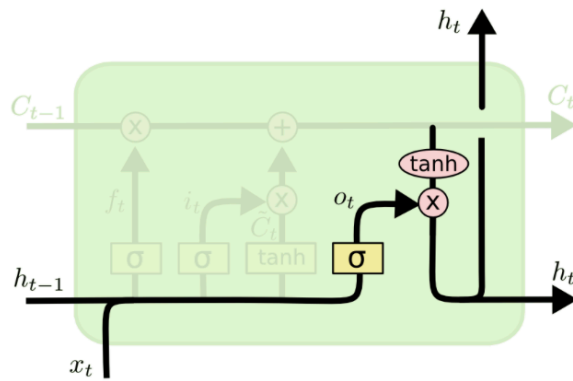
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM: cell state update



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

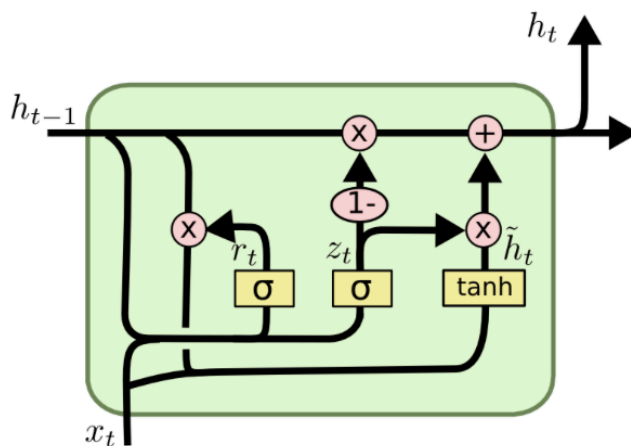
LSTM: hidden state update



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

GRU



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

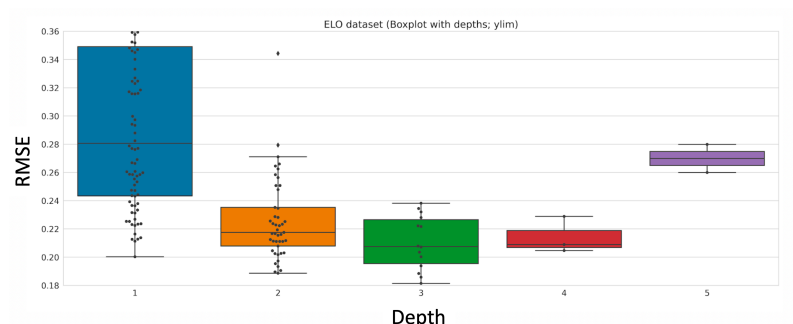
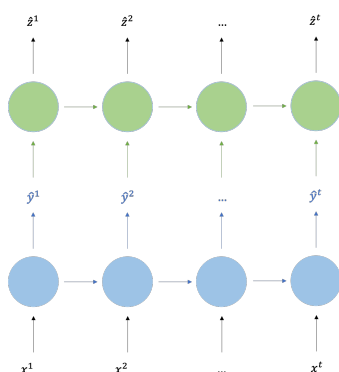
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

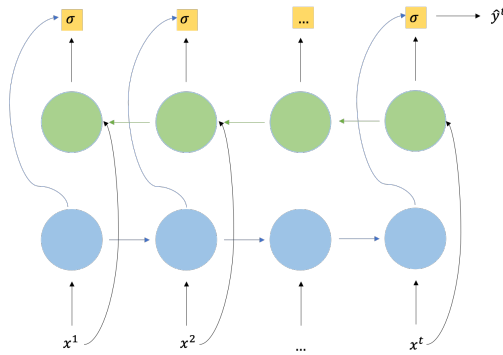
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

There is no cell state in GRU, and there are only two gates: update and reset. It is computationally simpler but very close in terms of results.

Multilayer LSTM



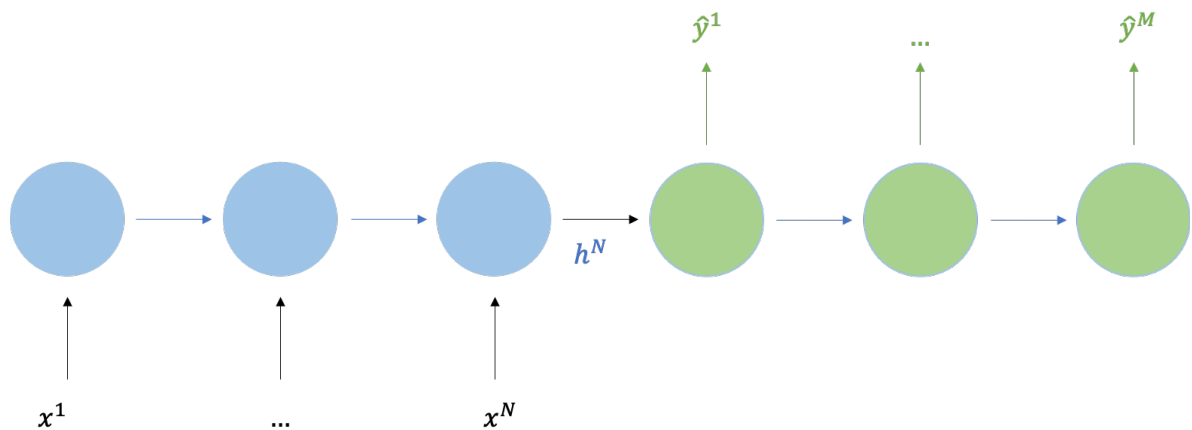
Bidirectional LSTM



Bidirectional LSTM are useful when we benefit from the future data or can use it

- Handwriting Recognition
- Speech Recognition
- Protein Structure Prediction (Bioinformatics)

Problems with LSTM



- The bottleneck for the representation
- Goes from past to future
- Long time to train, no improvement with GPU
- Tend to focus on the end of the sequence

Webinar

```
In [1]: import torch
import torch.nn as nn
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime
from tqdm.notebook import tqdm
```

```
In [2]: df = pd.read_csv('PJME_hourly.csv')
```

```
In [3]: df = df.set_index(["Datetime"]).rename(columns={'PJME_MW': 'value'})
df.index = pd.to_datetime(df.index)
df = df.sort_index()
```

```
In [4]: df.head()
```

Out [4]:

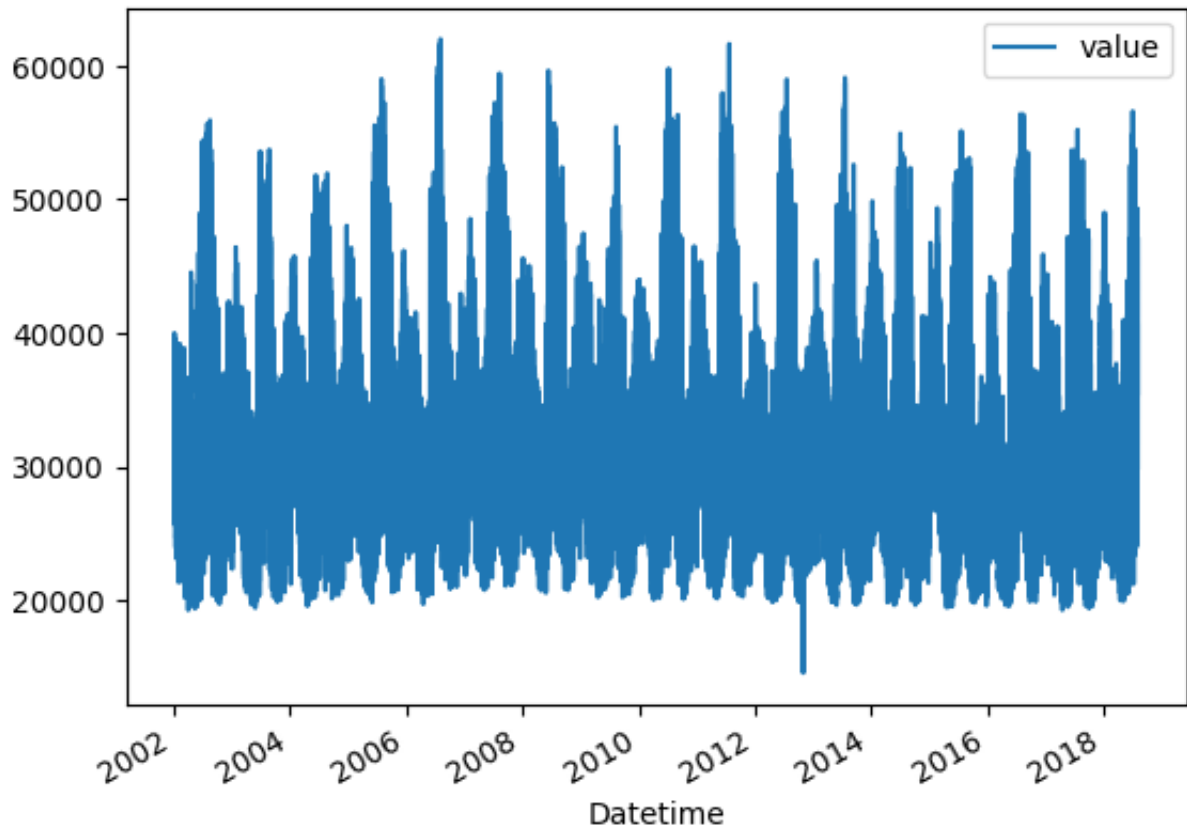
	value
Datetime	
2002-01-01 01:00:00	30393.0
2002-01-01 02:00:00	29265.0
2002-01-01 03:00:00	28357.0
2002-01-01 04:00:00	27899.0
2002-01-01 05:00:00	28057.0

In [5]: `df.tail()`

Out [5]:

	value
Datetime	
2018-08-02 20:00:00	44057.0
2018-08-02 21:00:00	43256.0
2018-08-02 22:00:00	41552.0
2018-08-02 23:00:00	38500.0
2018-08-03 00:00:00	35486.0

In [6]: `df.plot()`Out [6]: `<Axes: xlabel='Datetime'>`



```
In [7]: for k in range(1, 101):  
        df[f"lag_{k}"] = df["value"].shift(k)  
        df = df.copy()
```

/var/folders/33/j0cl7y453td68qb96j7bqcj4cf41kc/T/ipykernel_7917/2245740435.py:2: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using `pd.concat(axis=1)` instead. To get a de-fragmented frame, use `newframe = frame.copy()`

```
df[f"lag_{k}"] = df["value"].shift(k)
```

```
In [8]: df.head()
```

Out [8]:

	value	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7	lag_8
Datetime									
2002-01-01 01:00:00	30393.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 02:00:00	29265.0	30393.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 03:00:00	28357.0	29265.0	30393.0	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 04:00:00	27899.0	28357.0	29265.0	30393.0	NaN	NaN	NaN	NaN	NaN
2002-01-01 05:00:00	28057.0	27899.0	28357.0	29265.0	30393.0	NaN	NaN	NaN	NaN

5 rows × 101 columns

In [9]: `df = df.iloc[100:]`

```
In [15]: df["hour"] = df.index.hour
df["day"] = df.index.day
df["month"] = df.index.month
df["day_of_week"] = df.index.dayofweek
```

```
In [16]: for col, st, freq in zip(
    ["hour", "day", "month", "day_of_week"],
    [0, 0, 1, 0],
    [24, 365, 12, 7],
):
    df[f"sin_{col}"] = df[col].transform(lambda x: np.sin(2 * np.pi * (x
df[f"cos_{col}"] = df[col].transform(lambda x: np.cos(2 * np.pi * (x
```

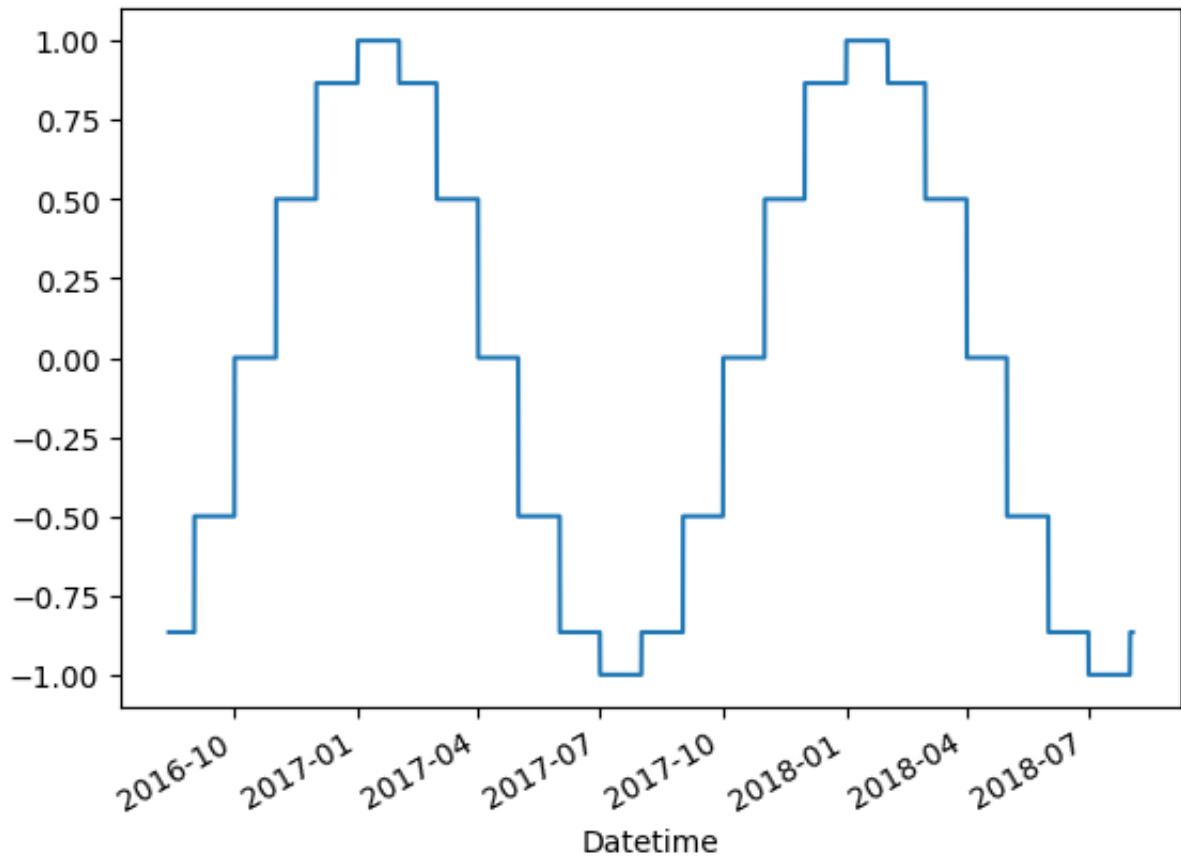
In [17]: `df`

Out[17]:

	value	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7
Datetime								
2002-01-05 05:00:00	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0	33202.0	35368.0
2002-01-05 06:00:00	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0	33202.0
2002-01-05 07:00:00	28557.0	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0
2002-01-05 08:00:00	29709.0	28557.0	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0
2002-01-05 09:00:00	31241.0	29709.0	28557.0	27399.0	26822.0	26669.0	27034.0	27501.0
...	...	...	...	...	...	...	...	...
2018-08-02 20:00:00	44057.0	45641.0	46760.0	46816.0	46989.0	47154.0	46534.0	45372.0
2018-08-02 21:00:00	43256.0	44057.0	45641.0	46760.0	46816.0	46989.0	47154.0	46534.0
2018-08-02 22:00:00	41552.0	43256.0	44057.0	45641.0	46760.0	46816.0	46989.0	47154.0
2018-08-02 23:00:00	38500.0	41552.0	43256.0	44057.0	45641.0	46760.0	46816.0	46989.0
2018-08-03 00:00:00	35486.0	38500.0	41552.0	43256.0	44057.0	45641.0	46760.0	46816.0

145266 rows × 113 columns

In [20]: `df.iloc[-24 * 30 * 24:]["cos_month"].plot()`Out[20]: `<Axes: xlabel='Datetime'>`



```
In [21]: import holidays
```

```
In [22]: us_holidays = holidays.US()
```

```
In [23]: df["is_holiday"] = df.index.to_series().transform(lambda x: int(x.replace
```

```
In [24]: from sklearn.model_selection import train_test_split
```

```
In [25]: df.to_parquet("features.parquet")
```

```
In [26]: X, y = df.drop("value", axis=1), df[["value"]]
```

```
In [27]: X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, sh
```

```
In [28]: from sklearn.preprocessing import StandardScaler
```

```
In [29]: X_train.head()
```

Out[29]:

	lag_1	lag_2	lag_3	lag_4	lag_5	lag_6	lag_7	lag_8
Datetime								
2002-01-05 05:00:00	26669.0	27034.0	27501.0	28635.0	30924.0	33202.0	35368.0	36762.0
2002-01-05 06:00:00	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0	33202.0	35368.0
2002-01-05 07:00:00	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0	33202.0
2002-01-05 08:00:00	28557.0	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0	30924.0
2002-01-05 09:00:00	29709.0	28557.0	27399.0	26822.0	26669.0	27034.0	27501.0	28635.0

5 rows × 113 columns

```
In [30]: xscaler = StandardScaler()
X_train = xscaler.fit_transform(X_train)
X_val = xscaler.transform(X_val)

yscaler = StandardScaler()
y_train = yscaler.fit_transform(y_train)
y_val = yscaler.transform(y_val)
```

```
In [33]: X_train[:5, :5]
```

```
Out[33]: array([[ -0.8792073 , -0.8224797 , -0.74990436, -0.5736884 , -0.2180048
  ],
  [ -0.85543237, -0.87919751, -0.8224719 , -0.74990147, -0.5736951
  ],
  [ -0.76577137, -0.85542265, -0.87918956, -0.82246893, -0.7499086
  ],
  [ -0.58582779, -0.7657619 , -0.85541476, -0.87918654, -0.8224762
  ],
  [ -0.40681657, -0.58581885, -0.76575423, -0.85541176, -0.8791940
  ]])
```

```
In [34]: from torch.utils.data import TensorDataset, DataLoader, Dataset

class SeqDataset(Dataset):
    def __init__(self, tensor, y, length):
        super().__init__()
        self.l = len(tensor)
```

```

        self.tensor = torch.cat((
            torch.zeros((length + 1, tensor.shape[1])),
            tensor
        ))
        self.y = torch.cat((
            torch.zeros((length + 1, y.shape[1])),
            y
        ), )
        self.length = length

    def __len__(self):
        return self.l

    def __getitem__(self, idx):
        x = self.tensor[idx: idx + self.length]
        y = self.y[idx + self.length]
        return x, y

```

In [35]: `bs = 64`

In [36]: `X_train = torch.Tensor(X_train)`
`X_val = torch.Tensor(X_val)`
`y_train = torch.Tensor(y_train)`
`y_val = torch.Tensor(y_val)`

In [37]: `ds_train = SeqDataset(X_train, y_train, 10)`
`ds_val = SeqDataset(X_val, y_val, 10)`

In [38]: `dl_train = DataLoader(ds_train, batch_size=bs, shuffle=False, drop_last=True)`
`dl_val = DataLoader(ds_val, batch_size=bs, shuffle=False, drop_last=True)`

In [39]: `class MyModel(nn.Module):`
 `def __init__(self, input_dim, hidden_dim, num_layers, output_dim, dropout_p):`
 `super(MyModel, self).__init__()`
 `self.hidden_dim = hidden_dim`
 `self.rnn = nn.LSTM(`
 `input_size=input_dim,`
 `hidden_size=hidden_dim,`
 `num_layers=num_layers,`
 `batch_first=True,`
 `dropout=dropout_p`
 `)`
 `self.fc = nn.Linear(in_features=hidden_dim, out_features=output_dim)`
 `def forward(self, x):`
 `out, (hn, cn) = self.rnn(x)`
 `out = out[:, -1, :]`

```
        out = self.fc(out)

    return out
```

```
In [40]: X_train.shape
```

```
Out[40]: torch.Size([116212, 113])
```

```
In [42]: y_train.shape
```

```
Out[42]: torch.Size([116212, 1])
```

```
In [43]: model = MyModel(
            input_dim=113,
            hidden_dim=64,
            output_dim=1,
            num_layers=3,
            dropout_p=0.1
        )
```

```
In [44]: import torch.optim as optim
```

```
In [45]: optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-6)
```

```
/Users/nstulov/miniconda3/envs/msai/lib/python3.12/site-packages/tqdm/aut
o.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywi
dgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
```

```
In [46]: lossf = nn.MSELoss()
```

```
In [47]: from tqdm.auto import tqdm
```

```
In [48]: for epoch in range(10):
            model.train()

            losses = []
            for batch, y in dl_train:
                loss_value = lossf(model(batch), y)
                loss_value.backward()

                optimizer.step()
                optimizer.zero_grad()

                losses.append(loss_value.item())

            model.eval()
            val_losses = []
            with torch.no_grad():
                for batch, y in dl_val:
                    loss_value = lossf(model(batch), y)
                    val_losses.append(loss_value.item())
```

```
print(
    f"[{epoch}/10] Training loss: {np.mean(losses):.4f}\t Validation
")
```

```
[0/10] Training loss: 0.0991    Validation loss: 0.1194
[1/10] Training loss: 0.0271    Validation loss: 0.0349
[2/10] Training loss: 0.0199    Validation loss: 0.0294
[3/10] Training loss: 0.0155    Validation loss: 0.0239
[4/10] Training loss: 0.0133    Validation loss: 0.0166
[5/10] Training loss: 0.0126    Validation loss: 0.0125
[6/10] Training loss: 0.0120    Validation loss: 0.0144
[7/10] Training loss: 0.0106    Validation loss: 0.0122
[8/10] Training loss: 0.0103    Validation loss: 0.0112
[9/10] Training loss: 0.0101    Validation loss: 0.0104
```

```
In [49]: model.eval()
         outputs = []
         true_vals = []
         with torch.no_grad():
             for batch, y in dl_val:
                 out = model(batch)
                 outputs.append(out.squeeze().numpy())
                 true_vals.append(y.squeeze().numpy())
```

```
In [54]: outputs = np.concatenate(outputs, axis=0)
```

```
In [56]: true_vals = np.concatenate(true_vals, axis=0)
```

```
In [59]: true_vals.shape
```

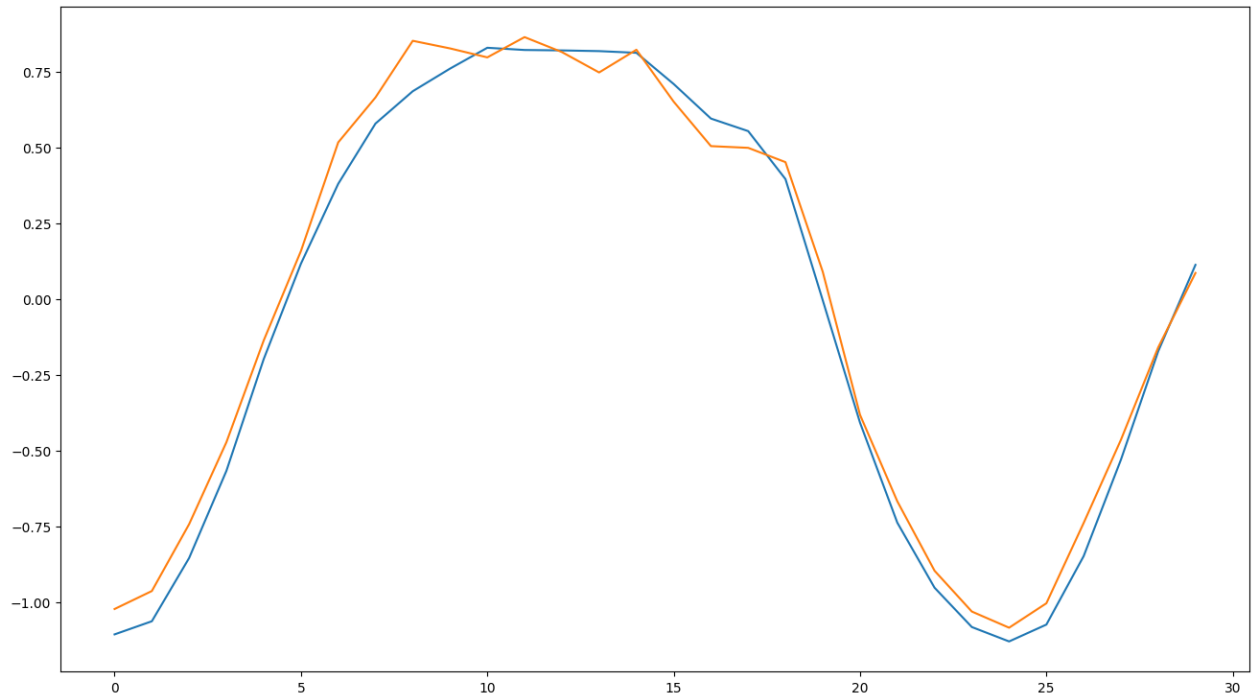
```
Out[59]: (28992,)
```

```
In [58]: y_val.shape
```

```
Out[58]: torch.Size([29054, 1])
```

```
In [63]: fig, ax = plt.subplots(figsize=(16,9))
         ax.plot(true_vals[-30:], label="ground truth")
         ax.plot(outputs[-30:], label="predictions")
```

```
Out[63]: [<matplotlib.lines.Line2D at 0x2faef02f0>]
```



```
In [64]: from sklearn.metrics import mean_squared_error
```

```
In [66]: mean_squared_error(true_vals, outputs)
```

```
Out[66]: 0.010381246
```

```
In [67]: np.random.randint(len(ds_val))
```

```
Out[67]: 8430
```

```
In [68]: ds_val[8430]
```

```
Out[68]: (tensor([[ -1.7399, -1.7940, -1.7856, ..., -1.3791, -0.3147, -0.1763],
                  [ -1.6038, -1.7399, -1.7940, ..., -1.3791, -0.3147, -0.1763],
                  [ -1.4047, -1.6038, -1.7399, ..., -1.3791, -0.3147, -0.1763],
                  ...,
                  [ -1.0949, -1.0155, -0.9706, ..., -1.3791, -0.3147, -0.1763],
                  [ -1.1962, -1.0949, -1.0155, ..., -1.3791, -0.3147, -0.1763],
                  [ -1.3019, -1.1962, -1.0949, ..., -1.3791, -0.3147, -0.1763]]),
          tensor([ -1.3939]))
```

```
In [70]: xx, yy = ds_val[8430]
```

```
In [71]: xx.shape
```

```
Out[71]: torch.Size([10, 113])
```

```
In [72]: xx = xx.clone()
          xx.requires_grad = True
          yy = yy.clone()

          zz = model(xx.unsqueeze(0))
```

```
In [73]: zz.shape
```

```
Out[73]: torch.Size([1, 1])
```

```
In [74]: loss_value = lossf(zz, yy.unsqueeze(0))

loss_value.backward()
gradients = xx.grad

gradients.shape
```

```
Out[74]: torch.Size([10, 113])
```

```
In [75]: loss_value.shape
```

```
Out[75]: torch.Size([])
```

```
In [76]: import seaborn as sns

sns.set(style="whitegrid", font_scale=1.5)
sns.despine()
```

<Figure size 640x480 with 0 Axes>

```
In [83]: from matplotlib.colors import SymLogNorm
```

```
In [87]: sns.heatmap((gradients ** 2).numpy(), cmap="RdBu", norm=LogNorm())
```

```
Out[87]: <Axes: >
```

